

Calibrated Partial Resets: Preventing Policy Collapse in Continual Reinforcement Learning

Luc McCutcheon¹, Evangelos Chatzaroulas², Saber Fallah¹

{lm01065, e.chatzaroulas, s.fallah}@surrey.ac.uk

¹Department of Mechanical Engineering, University of Surrey, United Kingdom

²Department of Computing Science, University of Surrey, United Kingdom

Abstract

Neural networks are hindered by accumulating dormant neurons and loss of expressivity throughout training, particularly in non-stationary data settings, such as continual supervised and reinforcement learning. Recently, neuron resets have been used to maintain gradient flow and restore plasticity. However, full unit reinitialization often sacrifices peak performance and can destabilize training, leading to policy collapse. To preserve plasticity without destabilizing training, we propose Calibrated Partial Resets (CPR), an optimizer that periodically pulls low-utility neurons toward their initialization, with pull strength scaled by each neuron’s utility. Unlike binary reset methods, *partial* resets avoid brittleness; unlike uniform decay, *calibrated* utility-scaling concentrates adjustment on the units that need it most. Among compared methods, only CPR avoids policy collapse over 400M training steps in SlipperyAnt, and it outperforms prior decay and reset-based methods on Continual MetaWorld and Continual MinAtar benchmarks. Ablations reveal a tunable trade-off between plasticity and peak performance, highlighting utility-scaled reinitialization as a promising direction for continual learning.

1 Introduction

Deep neural networks perform well in stationary training regimes but often lose plasticity under prolonged non-stationary training, limiting continual supervised learning and reinforcement learning (RL) agents that must adapt over long horizons. In RL, plasticity loss is especially consequential because policy updates change the agent’s future data distribution; small optimization instabilities can therefore compound through feedback loops into catastrophic policy degradation (Juliani & Ash, 2024).

Prior work links plasticity loss to decreasing feature rank, rising parameter norms, and representation collapse (Lyle et al., 2022). Sokar et al. (2023) show that training distribution shifts cause an increase in previously active units becoming dormant, Dohare et al. (2021; 2024) additionally show that decreases in stable rank correlate with reduced adaptation ability, and Lyle et al. (2023) posit that unstable loss landscapes and optimization difficulties contribute to plasticity loss.

One remedy is to adaptively reinitialize low-utility neuron units during training, thereby restoring unused network capacity. Existing methods differ primarily in how they estimate utility. Continual Backpropagation (CBP) (Dohare et al., 2021; 2024) defines a unit’s utility as the running average of its activation weighted by its outgoing connections, and subsequently zeros those connections upon reinitialization. ReDo (Sokar et al., 2023) uses batch-level activation statistics, while ReGraMa (Liu et al., 2025) uses gradient magnitude. Despite these differences, these methods share the same intervention structure: a unit below a threshold is fully reinitialized, while all other units are left unchanged.

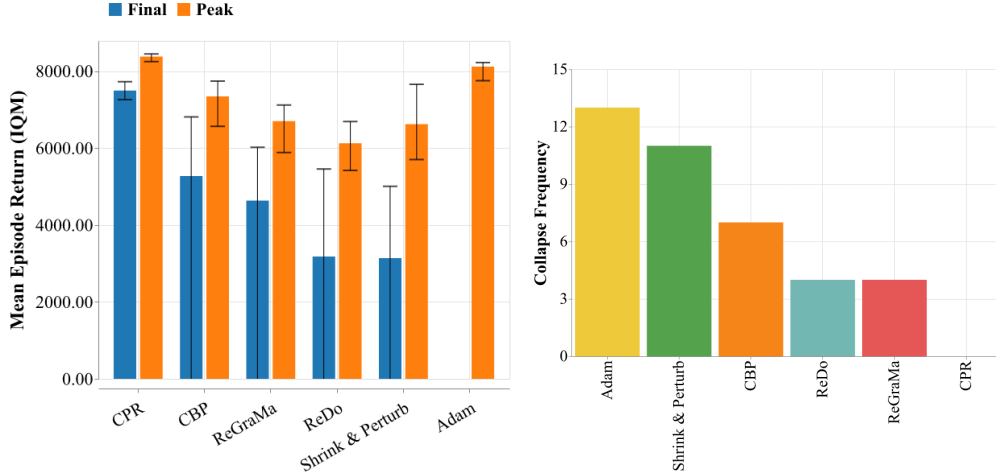


Figure 1: **Left:** Peak vs. final Interquartile Mean (IQM) episodic return in SlipperyAnt with Interquartile Range (IQR) error bars. **Right:** number of seeds with policy collapses. CPR achieves the highest return and **is the only method with zero policy collapses across all 15 seeds over 400M steps** while Adam collapses and receives a mean episodic return below zero

We argue that this conflates two design choices that should be separated: which units should be refreshed, and how strongly they should be changed. Binary reset methods are selective but discontinuous, introducing abrupt parameter changes that can destabilize long-horizon RL training. Uniform decay methods, such as Shrink & Perturb (Ash & Adams, 2020), make smoother changes but apply them indiscriminately across units. This exposes a missing design point: a method that is both selective and smooth.

To address this, we propose Calibrated Partial Resets (CPR), which periodically pulls neurons toward their initialization by a continuous coefficient determined by normalized per-neuron utility. Low-utility units are refreshed more strongly, while high-utility units are perturbed only weakly. Thus, CPR retains the selectivity of neuron-reset methods while avoiding the all-or-nothing updates that can make full reinitialization brittle in continual RL.

Our contributions can be summarized as follows:

- We identify binary neuron resets as a source of instability in long-horizon continual RL and propose CPR, a utility-scaled partial reset optimizer.
- We show that CPR avoids policy collapse over 400M steps in SlipperyAnt and improves performance on SlipperyHumanoid, Continual MetaWorld, and Continual MinAtar.
- We ablate reset magnitude and transformation shape, showing that partial, utility-weighted resets control the plasticity–stability tradeoff.

2 Background

We consider the Continual RL setting, where an agent operates in a sequence of N Markov decision processes (MDPs) $\{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_N\}$, each defined by the standard tuple $\mathcal{M}_k = (\mathcal{S}, \mathcal{A}, \mathcal{P}_k, \mathcal{R}_k, \gamma)$. The agent follows a stochastic policy $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ and aims to maximize the discounted return $\mathbb{E}_{a_t \sim \pi} [\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t)]$. While RL agents face inherent non-stationary training dynamics in various forms even in standard MDPs (Igl et al., 2020), we focus on problems with added non-stationarity imposed on the environment itself such that the agent must adapt over time. Given a training horizon $\mathcal{T}$, the environment switches to the next MDP every $\frac{\mathcal{T}}{N}$ timesteps, requiring the agent to continually adapt to changing dynamics.

2.1 Network Plasticity

We consider feed-forward networks with layers indexed by $\ell = 1, \dots, L$. Let $W^\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $b^\ell \in \mathbb{R}^{n_\ell}$ denote the parameters that map $h^{\ell-1}$ to pre-activations $z^\ell = W^\ell h^{\ell-1} + b^\ell$ and activations $h^\ell = \sigma(z^\ell)$. By convention, the *incoming* weights for unit i at layer ℓ are the i -th row of W^ℓ and the *outgoing* weights are the i -th column of $W^{\ell+1}$. We use mini-batches $\mathcal{B}_t$ at update step t and write $\mathbb{E}_{x \in \mathcal{B}_t}[\cdot]$ for batch averages.

Dormant-neuron ratio (Sokar et al., 2023). A neuron is *dormant* if its mean activation is small relative to the layer average.

$$\text{DormantRatio}_\ell = \frac{1}{n_\ell} \sum_{i=1}^{n_\ell} \mathbf{1} \left[\frac{\mathbb{E}_{x \sim \mathcal{B}_t} |h_i^\ell(x)|}{\frac{1}{n_\ell} \sum_{k=1}^{n_\ell} \mathbb{E}_{x \sim \mathcal{B}_t} |h_k^\ell(x)|} < \tau \right]. \quad (1)$$

where $\tau = 0.1$ in experiments.

Linearized-neuron ratio (Lyle et al., 2024). We define the number of *linearized units* per layer as the percentage of neurons for which the pre-activation value of a given neuron was positive. We can define linearization as:

$$\text{LinearizedRatio}_\ell = \frac{1}{n_\ell} \sum_{i=1}^{n_\ell} \mathbf{1} [\mathbb{E}_{x \in \mathcal{B}_t} [\mathbf{1}[z_i^\ell(x) > 0]] > \theta] \quad (2)$$

where $\theta = 0.9$ in experiments.

These metrics complement each other: Equation 1 captures under-used channels, while Equation 2 captures loss of gating diversity that reduces rank and gradient flow. We report both alongside accuracy/return. Note that these definitions are for rectifier-like activation functions only. For non-rectifier activations, an analogous variance-based criterion can be used (Lyle et al., 2024).

2.2 Unit Utility and Selective Reset

A *neuron-utility score* $S_i^\ell(t)$ quantifies how much unit i at layer ℓ contributes to learning and can be computed given network weights W^ℓ and activations h^ℓ or averaged gradients $\mathbb{E}_{x \in \mathcal{B}_t} [\nabla_{W_i^\ell} \mathcal{L}(W^\ell)]$ where $\mathcal{L}$ is a loss function. To stabilize selection under non-stationarity, it is useful to compute a layer-normalized exponential moving average (EMA) of the score $u_i^\ell(t)$:

$$\begin{aligned} u_i^\ell(t) &\leftarrow \beta u_i^\ell(t-1) + (1-\beta) \tilde{S}_i^\ell(t), \\ \tilde{S}_i^\ell(t) &= \frac{S_i^\ell(t)}{\frac{1}{n_\ell} \sum_{k=1}^{n_\ell} S_k^\ell(t)}, \quad \beta \in [0, 1). \end{aligned} \quad (3)$$

where β is the EMA decay parameter. Normalization over layers (through dividing by the layer mean) makes selection thresholds comparable across layers and training time; see Figure 11 for a visualization.

Selective reset methods. We study neuron-reset methods that operate immediately after the base optimizer update step and reinitialize units with low utility score S_i^ℓ or u_i^ℓ based on a *reset rule* $\mathfrak{R}$.

2.3 Training Instability in Continual Reinforcement Learning

Existing neuron-reset methods (e.g., CBP, ReDo, ReGraMa (Dohare et al., 2024; Sokar et al., 2023; Liu et al., 2025)) differ in utility estimation and reset scheduling, but share a *binary* intervention $\mathfrak{R}(S_i^\ell, \tau) = \mathbf{1}[S_i^\ell(t) < \tau]$, which fully reinitializes low-utility neurons

$$\begin{aligned} W_{i,:}^\ell &\leftarrow (1 - \mathfrak{R}(S_i^\ell, \mathfrak{r})) W_{i,:}^\ell + \mathfrak{R}(S_i^\ell, \mathfrak{r}) \xi_{i,:}^\ell \\ W_{:,i}^{\ell+1} &\leftarrow (1 - \mathfrak{R}(S_i^\ell, \mathfrak{r})) W_{:,i}^{\ell+1} \end{aligned} \quad (4)$$

where $\xi_{i,:}^\ell \sim \mathcal{D}_{\text{init}}$, with $\mathcal{D}_{\text{init}}$ being a weight initialization distribution, for instance Kaiming (He et al., 2015) or Xavier (Glorot & Bengio, 2010). $W_{i,:}^\ell$ denotes the incoming weights into unit i while $W_{:,i}^{\ell+1}$ are the outgoing weights.

Notably, under this reset rule, neurons either get reset if they are deemed low-utility, in which case their incoming weights are completely reinitialized, or they do not. This all-or-nothing reinitialization introduces sudden, large weight changes which are a known source of instability in deep RL (Dohare et al., 2023) and can cause gradient spikes that destabilize training. Additionally, units that are reinitialized have their outgoing weights zeroed, which can lead to a distribution shift in the inputs of the next layer, causing further training instability, especially in the case where layer normalization (Ba et al., 2016) is not used.

We empirically find that these instabilities can and do arise in these algorithms in continual RL tasks in Figure 4 and can also lead to policy collapse in Figure 1. Typical policy collapse mitigation such as early stopping (Li et al., 2024) and gradient stabilization approaches such as learning rate annealing (Loshchilov & Hutter, 2016) are designed for stationary, single-task settings and cannot be readily applied to continual learning where ideally the agent should be able to learn throughout its lifetime up to an infinite horizon.

3 Calibrated Partial Resets (CPR)

Utility score estimation. Following recent work showing gradient-based utility criteria scale more favorably (Liu et al., 2025; Hernandez-Garcia et al., 2025), we define the raw utility score for unit i in layer ℓ as the mean gradient magnitude of its incoming weights:

$$S_i^\ell(t) = \mathbb{E}_{x \in \mathcal{B}_t} \left[\left\| \nabla_{W_{i,:}^\ell} \mathcal{L}(x) \right\| \right] \quad (5)$$

where $\mathcal{L}$ is the loss function being optimized. This score is then layer-normalized and smoothed via the EMA defined in Equation 3 to obtain u_i^ℓ .

Reset Mechanism. CPR differs from prior work along two axes. Binary reset methods (subsection 2.3) fully reinitialize selected units, which introduces abrupt parameter changes at reset events. Decay-based methods such as Shrink & Perturb (Ash & Adams, 2020) apply a uniform shrinkage to all units, ignoring per-neuron utility. CPR is *partial* on one axis and *calibrated* on the other: every f steps, each unit’s weights are pulled toward initialization by a coefficient $r_i^\ell = \rho \phi(u_i^\ell) \in [0, \rho]$, where ϕ is a monotonically decreasing function of the unit’s utility and $\rho \in (0, 1]$. We use *calibrated* to emphasize that the reset magnitude is set per-neuron from a measured signal, rather than applied uniformly (decay) or as an all-or-nothing event (binary reset).

The two design choices contribute roughly independently. Avoiding full reinitialization reduces the gradient spikes that destabilize binary-reset methods, regardless of how aggressively low-utility units are targeted. Targeting via utility even coarsely, e.g. by resetting the below-mean-utility units in each layer, strongly outperforms uniform shrinkage. We map utility to a reset intensity via the shape function

$$\phi(u_i^\ell) = \min(2\sigma[-\kappa(u_i^\ell - 1)], 1), \quad (6)$$

where u_i^ℓ is the per-layer normalized utility with mean 1; σ is the logistic sigmoid function $\sigma(x) = (1 + e^{-x})^{-1}$; and $\kappa > 0$ determines the sharpness of the mapping. The factor of 2 anchors the curve at the layer mean ($\phi(u_i^\ell) = 1$ when $u_i^\ell = 1$). Units with above average utility receive progressively smaller resets, while those below the mean receive the maximum per-unit reset fraction ρ (alternative smooth shapes have a measurable but secondary effect; see Appendix G and Figure 10).

Notably, as $\kappa \rightarrow \infty$, ϕ approaches a step function: $\lim_{\kappa \rightarrow \infty} \phi(u) = \mathbf{1}[u \leq 1]$, applying resets only to units below the layer-average utility. Even in this limit, CPR remains partial: the below-mean-utility units in each layer are pulled toward initialization by ρ rather than fully reinitialized. This isolates the contribution of partial-vs-full reset from that of utility-based targeting. At the opposite extreme, $\kappa = 0$ applies uniform resets to all neurons. Empirically, our sharpness ablation (Figure 12) shows that CPR is robust across a range of $\kappa \in [2, 20]$, with performance degrading only at small $\kappa \leq 1$ values where the mapping becomes nearly uniform and the per-neuron signal is lost. Given this wide plateau, we fix $\kappa = 16$ across all experiments rather than treating it as a tunable hyperparameter.

The resulting partial reset operator for unit i in layer ℓ is

$$\begin{aligned} W_{i,:}^{\ell} &\leftarrow (1 - r_i^{\ell}) W_{i,:}^{\ell} + r_i^{\ell} \xi_{i,:}^{\ell}, \\ W_{:,i}^{\ell+1} &\leftarrow (1 - r_i^{\ell}) W_{:,i}^{\ell+1}, \end{aligned} \tag{7}$$

where, as in Equation 4, $\xi_{i,:}^{\ell} \sim \mathcal{D}_{\text{init}}$. We perform CPR updates every update frequency f and compute the EMA of utility scores between update steps only. This is implemented by using data from every optimization step to compute the utility score EMA, but along with CPR updates, we reset utility running averages to their mean 1. Algorithm 1 in Appendix A gives the complete update algorithm, including utility-EMA accumulation, reset scheduling and utility re-centering.

4 Results

Experiment setup. Our benchmark suite includes long-horizon friction-shift control tasks (SlipperyAnt and SlipperyHumanoid), continual off-policy control (Continual MetaWorld), and continual visual control (Continual MinAtar), with appendix experiments in single-task RL and continual supervised learning.

We evaluate CPR against recent continual learning methods in two challenging continual RL settings: SlipperyAnt and SlipperyHumanoid. These are modified Brax-based variants of the OpenAI Gym Ant and Humanoid environments (Brockman et al., 2016; Freeman et al., 2021), similar to the continual RL setting studied by Dohare et al. (2024). SlipperyAnt¹ requires a running ant-like robot to adapt to global friction changes, while SlipperyHumanoid requires a running humanoid to adapt as ground friction changes periodically. This challenges continual learning agents, as they must retain locomotion ability while selectively forgetting friction dynamics in their implicit world model.

We train agents for 400 million steps, sample friction log-uniformly from $[0.02, 2.0]$ every 20 million steps and use 2048 parallel environments. We additionally open source the codebase for these experiments².

As the base RL algorithm we use Proximal Policy Optimization (PPO) (Schulman et al., 2017) with Adam (Kingma & Ba, 2014) as the base optimizer, with hyperparameters and network sizes provided in Appendix F.

SlipperyAnt and SlipperyHumanoid use 15 random seeds; MT1 uses 10 seeds with Continual MetaWorld and Continual MinAtar using 5 seeds. We provide further details for these benchmarks in Appendix H. We report the IQM across seeds, as proposed by Agarwal et al. (2021). Interquartile ranges (IQR) are provided in Appendix I (Tables 11 and 12). We selected the hyperparameters with highest average final return (Appendix F, Table 5).

¹While SlipperyAnt was studied in prior work (Dohare et al., 2024), implementing it on top of the Brax version leads to different rewards and dynamics.

²github.com/LucMc/continual-learning/

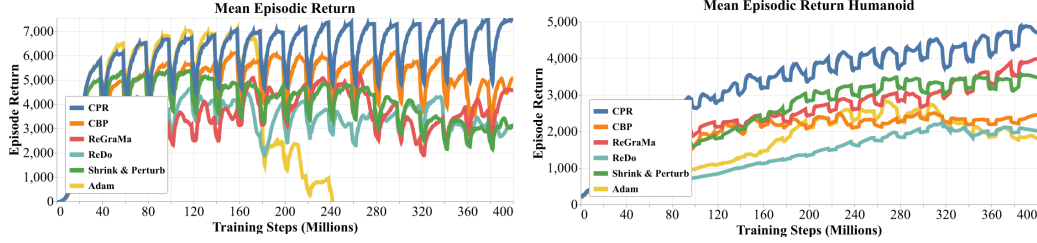


Figure 2: Comparison of IQM of episodic returns over 15 seeds. **Left:** SlippyAnt. **Right:** SlippyHumanoid. IQR is removed for visual clarity and can be found in Tables 11 and 12. CPR maintains high return in SlippyAnt while leading throughout training in SlippyHumanoid.

4.1 CPR Maintains or Improves Peak Performance

On SlippyAnt, we observe in Figure 2 that CPR can consistently achieve peak performance after each friction change. CPR maintains this ability even after 400 million steps of training. Adam with no neuron-reset method attached achieves strong peak performance but eventually collapses. Meanwhile, other reset-based methods such as ReDo and ReGraMa with a binary reset rule peak later on in training before themselves showing a decline in performance. This illustrates a plasticity-performance trade-off in this setting where existing methods either peak high and collapse, such as Adam which has no mechanism to maintain plasticity, or maintain trainability but are unable to learn optimal policies for each task due to unstable training dynamics arising from the neuron resets that help maintain gradient flow. In contrast, CPR can achieve both objectives.

In SlippyHumanoid (Figure 2), CPR substantially outperforms baselines in mean episodic return throughout training. This suggests more effective transfer across task changes.

We evaluate CPR in two further Continual RL benchmarks inspired by Wolczyk et al. (2021) and Tang et al. (2025), using Continual MetaWorld and Continual MinAtar respectively. Continual MetaWorld sequentially iterates through the MT10 taskset using SAC and Muon as the base optimizer. Continual MinAtar evaluates methods on a sequence of Atari tasks using a CNN architecture.

Table 1: Continual MetaWorld IQM of return over training steps and final performance using 5 seeds with IQR

Method	Avg	Final
CPR	0.231 ± 0.005	0.200 ± 0.010
CBP	0.204 ± 0.009	0.100 ± 0.030
ReDo	0.199 ± 0.017	0.137 ± 0.025
ReGraMa	0.196 ± 0.003	0.155 ± 0.007
Shrink & Perturb	0.199 ± 0.007	0.150 ± 0.025
Muon	0.179 ± 0.008	0.145 ± 0.028
Adam	0.147 ± 0.003	0.167 ± 0.023

Table 2: Continual MinAtar IQM of return over training steps and final performance using 5 seeds with IQR

Method	Avg	Final
CPR	75.6 ± 14.3	52.3 ± 25.0
CBP	65.6 ± 3.2	5.8 ± 14.9
ReDo	68.0 ± 7.6	11.5 ± 11.7
ReGraMa	65.5 ± 2.1	15.0 ± 13.6
Shrink & Perturb	61.1 ± 8.1	3.3 ± 2.6
Adam	54.9 ± 3.5	1.9 ± 0.8

4.2 CPR Improves Training Stability

Dormant and linearized neurons. Figure 3 shows that both CPR and CBP keep the dormant-neuron ratio nearly flat over 400M steps, whereas Adam and baselines that don’t use running statistics of the utility score rise steadily, with acceleration after ~ 200 – 300 M steps, which coincides with the return degradation in Figure 2. Methods that strongly suppress dormancy tend to have higher linearized-unit ratios initially; CPR and CBP maintain steady dormancy and linearization ratios, whereas other methods oscillate. Thus, CPR preserves plasticity-related statistics without the large late-training drift observed in several baselines.

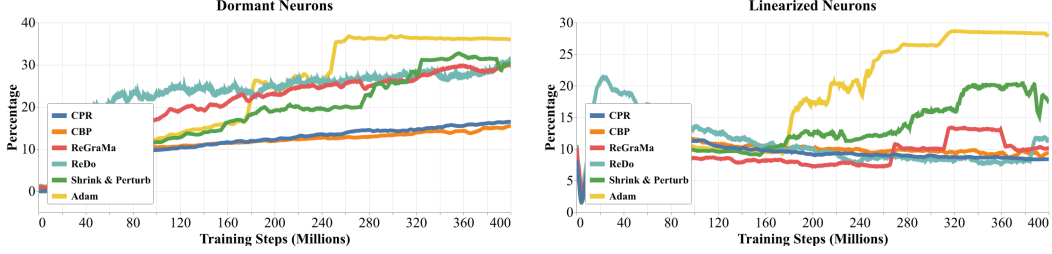


Figure 3: **Left:** Dormant-neuron ratio. **Right:** Linearized-neuron ratio. IQM across 15 seeds (lower is better). Lower dormancy correlates with higher plasticity and lower linearization correlates with higher representation capacity. This shows all methods reduce unit dormancy and linearization, though CBP and CPR appear most effective.

Gradient norm. Figure 4 reports actor gradient norms for SlippyAnt, with SlippyHumanoid gradient norms shown in Figure 14. CPR exhibits smaller fluctuations over time relative to baselines with binary reset rules, which aligns with its ability to avoid collapses and sustain high final return. Parameter-norm dynamics show a similar pattern (Figure 15): baseline methods exhibit higher parameter growth, which Dohare et al. (2024) identify as a key driver of plasticity loss, while CPR constrains parameter norms throughout training.

4.3 CPR Prevents Policy Collapse

SlippyAnt’s 400M-step horizon and periodic friction shifts make it our primary stress test for training stability: it is the setting in which the instabilities of binary reset methods manifest most visibly. We use it as a diagnostic for catastrophic policy degradation; the broader stability benefits of CPR, reflected in gradient and parameter norm dynamics (Figure 4, Figure 15) and in average performance on Continual MetaWorld and Continual MinAtar (Table 1, Table 2), extend beyond this setting.

In SlippyAnt, we measure a policy collapse as a sustained drop of at least 8,000 in episodic return from a prior peak, persisting for at least 4M timesteps. The 8,000 threshold spans the dynamic range of the environment (each method achieves a peak return of at least $\approx 6,000$, and episodic return is lower-bounded near $-2,000$); smaller thresholds risk classifying transient drops at task boundaries as collapses, while larger ones exclude methods whose peaks do not exceed the gap. The 4M-step persistence requirement excludes single-update spikes from natural variance.

The separation between CPR and reset-based baselines is not an artifact of this choice. Under a stricter 4,000 threshold, CPR remains the only method with zero collapses across all 15 seeds, while every binary-reset baseline and Adam exhibit collapses on more than 40% of runs. Under a lenient 10,000 threshold, CPR, ReDo, and ReGraMa each have zero collapses, but this threshold excludes any method whose peak return does not exceed it. CPR uniquely avoids collapse across the meaningful range of the threshold (Figure 1).

Controlling the stability–plasticity tradeoff. In CPR, ρ acts as a direct control over the stability–plasticity balance. Small values of ρ minimize the disruption caused by each reset, but in challenging environments such as SlippyAnt, they may fail to inject sufficient plasticity for sustained adaptation, leading to gradual performance degradation.

Conversely, large values of ρ increase plasticity, but can reduce peak and average performance by over-resetting useful structure. Despite this tradeoff, our sweeps reveal a robust operational range of $\rho \in [0.01, 0.05]$ in which CPR consistently prevents policy collapse across both SlippyAnt and SlippyHumanoid. Thus, while ρ can be tuned to optimize the stability–plasticity tradeoff for a given environment, collapse prevention is not brittle to precise hyperparameter selection. We empirically demonstrate the effect of ablating this parameter in SlippyAnt in Figure 5.

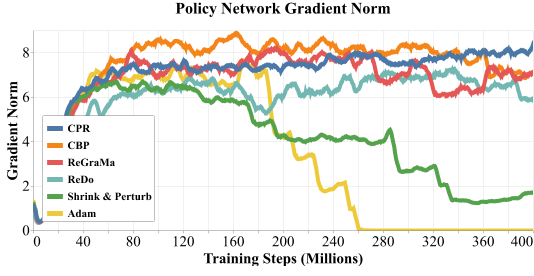


Figure 4: Actor gradient norm IQM over 15 seeds in SlippyAnt

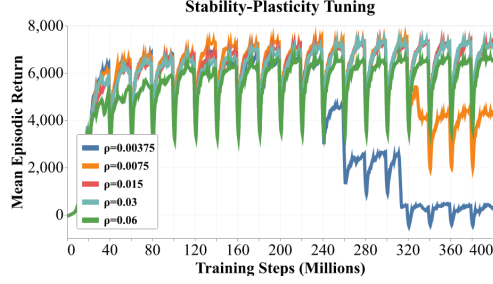


Figure 5: Tuning ablation for max per-unit reset ρ in SlippyAnt. A low ρ preserves stability but hinders adaptation, while high ρ values enhance plasticity but suppress peak reward

5 Discussion

Calibrated Partial Resets (CPR) fills the gap between uniform decay and binary neuron reinitialization: it preserves the selectivity of reset methods while replacing all-or-nothing updates with utility-scaled partial resets. Across SlippyAnt, SlippyHumanoid, Continual MetaWorld, and Continual MinAtar, CPR improves long-horizon continual RL stability and performance. In the 400M-step friction-shift setting, CPR is the only method to avoid policy collapse while recovering peak performance after task changes. Although the permuted MNIST experiments use more tasks than any other, the difficulty is low enough that all methods except Adam saturate the benchmark. Similarly, Continual MinAtar (Table 2) experiences less plasticity loss than SlippyAnt as it contains only 2 task transitions, highlighting the difficulty in designing and assessing plasticity loss benchmarks.

We attribute CPR’s collapse avoidance to utility-scaled partial resets, which inject plasticity without the sharp parameter updates induced by full reinitialization. This absence of collapse is a qualitative shift in training reliability, especially for long-horizon RL deployments where collapse may be difficult to recover from. Gradient, dormant-unit, linearization, and parameter-norm diagnostics provide convergent evidence for this mechanism, though no single diagnostic is decisive. The maximum per-unit reset fraction ρ provides direct control over the plasticity–performance tradeoff.

Limitations. While we observe a substantial improvement in training stability and performance in the continual RL setting, CPR introduces implementation complexity compared to simpler heuristics such as Shrink & Perturb. However, this is largely mitigated by our modular Optax implementation, which requires minimal modification to standard training pipelines. Additionally, calculating and storing per-neuron utility scores incurs a marginal computational overhead comparable to baselines; in our highest-dimensional setting, SlippyHumanoid, we measured a $\sim 6\%$ increase in wall-clock training time compared to Adam (see Table 13). Finally, CPR introduces three tunable hyperparameters, the same as baselines. For the maximum per-unit reset fraction ρ and decay rate β , we found that defaults of $\rho = 0.015$ and $\beta = 0.99$ generalize across our continuous control suite without task-specific tuning, though modest gains can be made by adjusting ρ per environment.

Our experiments focus primarily on non-stationarity induced by changing task dynamics and task identity. We do not claim that CPR resolves all forms of continual RL instability, such as reward non-stationarity, observation shifts, morphology changes, or large-scale visual control. Similarly, our diagnostics support but do not fully identify the causal pathway by which partial resets prevent collapse. Future work should isolate roles of utility estimation, partial reinitialization, optimizer-state handling, and normalization under a wider range of architectures and non-stationary processes.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 34:29304–29320, 2021.
- Jordan Ash and Ryan P Adams. On warm-starting neural network training. *Advances in neural information processing systems*, 33:3884–3894, 2020.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, june 2016. *arXiv preprint arXiv:1606.01540*, 2016.
- DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Antoine Dedieu, Claudio Fantacci, Jonathan Godwin, Chris Jones, Ross Hemsley, Tom Hennigan, Matteo Hessel, Shaobo Hou, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Markus Kunesch, Lena Martens, Hamza Merzic, Vladimir Mikulik, Tamara Norman, George Papamakarios, John Quan, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Laurent Sartran, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Miloš Stanojević, Wojciech Stokowiec, Luyu Wang, Guangyao Zhou, and Fabio Viola. The DeepMind JAX Ecosystem, 2020. URL <http://github.com/google-deepmind>.
- Shibhansh Dohare, Richard S Sutton, and A Rupam Mahmood. Continual backprop: Stochastic gradient descent with persistent randomness. *arXiv preprint arXiv:2108.06325*, 2021.
- Shibhansh Dohare, Qingfeng Lan, and A Rupam Mahmood. Overcoming policy collapse in deep reinforcement learning. In *Sixteenth European Workshop on Reinforcement Learning*, 2023.
- Shibhansh Dohare, J Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A Rupam Mahmood, and Richard S Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026): 768–774, 2024.
- Vivek F Farias and Adam D Jozefiak. Self-normalized resets for plasticity in continual learning. *arXiv preprint arXiv:2410.20098*, 2024.
- C Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax—a differentiable physics engine for large scale rigid body simulation. *arXiv preprint arXiv:2106.13281*, 2021.
- Robert M French. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 249–256. JMLR Workshop and Conference Proceedings, 2010.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. Pmlr, 2018.

-
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pp. 1026–1034, 2015.
- J Fernando Hernandez-Garcia, Shibhansh Dohare, Jun Luo, and Rich S Sutton. Reinitializing weights vs units for maintaining plasticity in neural networks. *arXiv preprint arXiv:2508.00212*, 2025.
- Maximilian Igl, Gregory Farquhar, Jelena Luketina, Wendelin Boehmer, and Shimon Whiteson. Transient non-stationarity and generalisation in deep reinforcement learning. *arXiv preprint arXiv:2006.05826*, 2020.
- Arthur Juliani and Jordan Ash. A study of plasticity loss in on-policy deep reinforcement learning. *Advances in Neural Information Processing Systems*, 37:113884–113910, 2024.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Saurabh Kumar, Henrik Marklund, and Benjamin Van Roy. Maintaining plasticity in continual learning via regenerative regularization. *arXiv preprint arXiv:2308.11958*, 2023.
- Hao Li, Gopi Krishnan Rajbahadur, Dayi Lin, Cor-Paul Bezemer, and Zhen Ming Jiang. Keeping deep learning models in check: A history-based approach to mitigate overfitting. *IEEE Access*, 12:70676–70689, 2024.
- Jiashun Liu, Zihao Wu, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, and Ling Pan. Measure gradients, not activations! enhancing neuronal activity in deep reinforcement learning. *arXiv preprint arXiv:2505.24061*, 2025.
- Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.
- Clare Lyle, Mark Rowland, and Will Dabney. Understanding and preventing capacity loss in reinforcement learning. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=ZkC8wKoLbQ7>.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 23190–23211. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/lyle23b.html>.
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, Hado van Hasselt, Razvan Pascanu, James Martens, and Will Dabney. Disentangling the causes of plasticity loss in neural networks. *arXiv preprint arXiv:2402.18762*, 2024.
- Reginald McLean, Evangelos Chatzaroulas, Luc McCutcheon, Frank Röder, Tianhe Yu, Zhanpeng He, KR Zentner, Ryan Julian, JK Terry, Isaac Woungang, et al. Meta-world+: An improved, standardized, rl benchmark. *arXiv preprint arXiv:2505.11289*, 2025.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

-
- Michał Nauman, Michał Bortkiewicz, Piotr Miłoś, Tomasz Trzcinski, Mateusz Ostaszewski, and Marek Cygan. Overestimation, overfitting, and plasticity in actor-critic: the bitter lesson of reinforcement learning. *arXiv preprint arXiv:2403.00514*, 2024.
- Chinmay Savadikar, Michelle Dai, and Tianfu Wu. Continual learning via learning a continual memory in vision transformer. *arXiv preprint arXiv:2303.08250*, 2023.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pp. 1889–1897. PMLR, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning. In *International Conference on Machine Learning*, pp. 32145–32168. PMLR, 2023.
- Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Hongyao Tang, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, and Glen Berseth. Mitigating plasticity loss in continual reinforcement learning by reducing churn. *arXiv preprint arXiv:2506.00592*, 2025.
- Hado Van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. *arXiv preprint arXiv:1812.02648*, 2018.
- Maciej Wolczyk, Michał Zajka, Razvan Pascanu, Lukasz Kucinski, and Piotr Miłos. Continual world: A robotic benchmark for continual reinforcement learning. *Advances in Neural Information Processing Systems*, 34:28496–28510, 2021.

A CPR Algorithm

Algorithm 1 CPR (utilities every step; resets every f steps)

Require: Update frequency f , EMA decay β , steepness κ , max reset ρ

```

1: Input: current step  $t$ 
2: Compute per-neuron utility  $S_i^\ell(t)$  {layer-normalized}
3: Update EMA:  $u_i^\ell \leftarrow \beta u_i^\ell + (1 - \beta) \tilde{S}_i^\ell$ 
4: if  $t \bmod f = 0$  and  $t > 0$  then {apply resets every  $f$  steps}
5:   for each layer  $\ell$  do
6:     for each neuron  $i$  do
7:        $r_i^\ell \leftarrow \rho \cdot \min(2 \times \sigma(-\kappa(u_i^\ell - 1)), 1)$  {See Equation 6}
8:        $W_{i,:}^\ell \leftarrow (1 - r_i^\ell) W_{i,:}^\ell + r_i^\ell \xi_{i,:}^\ell$ 
9:        $W_{:,i}^{\ell+1} \leftarrow (1 - r_i^\ell) W_{:,i}^{\ell+1}$ 
10:       $u_i^\ell \leftarrow 1$  {reset running utility after a reset step}
11:     end for
12:   end for
13: end if

```

B Additional Related Work

Plasticity Loss in Deep Learning and RL. Training stability is a central challenge in RL, arising from function approximation, bootstrapping, and off-policy updates (Sutton et al., 1998; Van Hasselt et al., 2018). Policy shifts can be amplified through feedback loops, spiraling into catastrophic performance collapse (Juliani & Ash, 2024). Accordingly, many major advances in RL have centered on improving stability including replay buffers (Mnih et al., 2013) and trust regions (Schulman et al., 2015). Neural networks under prolonged non-stationary training forget prior knowledge (French, 1999; Kirkpatrick et al., 2017) and lose their ability to adapt (Lyle et al., 2024; Nauman et al., 2024; Dohare et al., 2021), a phenomenon that has drawn increasing attention across class-incremental learning (Savadikar et al., 2023), supervised learning (Ash & Adams, 2020), and RL (Lyle et al., 2022). Dohare et al. (2024) links plasticity loss to rising parameter norms, dormant units, and reductions in stable rank. These findings motivate methods that restore trainability by controlling weight statistics or re-introducing randomness, including L2 regularization (Kumar et al., 2023), shrink & perturb (Ash & Adams, 2020), and binary reset methods (Sokar et al., 2023; Dohare et al., 2024; Liu et al., 2025).

Reinitialization Algorithms. A closer line of work preserves plasticity by periodically reinitializing low-utility units, motivated by work that demonstrates how plasticity loss can be attributed to only a subset of neurons that become dormant (Dohare et al., 2021; Sokar et al., 2023). Normalization (Ba et al., 2016) can mitigate plasticity loss, but prior work Hernandez-Garcia et al. (2025) suggests reinitialization provides additional benefit. CBP (Dohare et al., 2021; 2024) identifies inactive neurons by using moving averages of activation statistics and continually resets them. More recent work such as ReDo (Sokar et al., 2023) initializes dormant neurons periodically and uses activation statistics computed over a single batch of data, simplifying implementation over CBP without cost to performance. Alternative approaches such as SNR (Farias & Jozefiak, 2024) count the time between feature activations and ReGraMa (Liu et al., 2025) uses gradient magnitudes as a more scalable metric for unused neurons than activation statistics.

CPR uses gradient magnitude utilities, as in ReGraMa, but smooths them over multiple optimization steps, as in CBP. Additionally, unlike prior reset methods, CPR does not fully reinitialize a targeted subset of neurons, but instead partially reinitializes all neurons according to their utility. Our approach therefore sits between the uniform perturbations of decay-based methods (Ash & Adams, 2020) and binary reinitialization of neuron reset methods. Our 400M-step experiments extend beyond typical horizons in prior reset-method evaluations and expose long-horizon instabilities in binary reset methods that CPR avoids.

C Experiments in Standard RL

To assess whether the plasticity mechanisms in CPR induce instability or performance drops in stationary environments, we evaluate the method on the MetaWorld MT1 benchmark (McLean et al., 2025) using Soft Actor Critic (SAC) (Haarnoja et al., 2018). We chose SAC over PPO as it is standard for the benchmark and allows us to verify CPR’s performance under off-policy methods. We compare CPR against Adam and other reset-based baselines across 10 distinct manipulation tasks over 2 million training steps. Reset methods hyperparameters are reported in Table 7. We use the same SAC configuration as in Table 6, except that the single-task MT1 experiments use a 1M replay buffer and replay ratio 4; all methods are evaluated under the same MT1 settings.

C.1 Analysis of Results

The results, visualized in Figure 6, demonstrate that CPR generalizes effectively to stationary settings, imposing no performance penalty compared to the base optimizer. We observe a saturation in performance across methods, particularly on easier tasks: the 2M-step horizon is insufficient to induce catastrophic plasticity loss under standard Adam, so baselines appear similarly capable in

this regime. The differentiating benefits of CPR emerge primarily once task boundaries force rapid distribution shifts (Continual MetaWorld, Table 1) or training horizons extend into the regime where plasticity loss compounds into collapse (SlipperyAnt, 400M steps). We include MT1 results not as the principal evidence for CPR, but to verify the method imposes no penalty in stationary settings, a necessary property for a general-purpose continual learning optimizer.

Robustness and sample efficiency. On challenging tasks such as Peg Insert Side and Pick Place, CPR tracks the performance profile of standard Adam or exceeds it (Figure 6). This suggests that calibrated partial resets maintain the network’s effective rank without disrupting the policy during the critical exploration phases of training. Reset-based baselines like ReDo, which improve plasticity in the continual setting, occasionally exhibit minor performance drops here (e.g., Reach-v3), while CPR matches the stability of Adam, retaining the benefits of plasticity without the volatility of binary resets.

D Experiments in Continual Supervised Learning

We evaluate CPR in the Continual Permuted MNIST setting, a supervised learning regime where the input distribution is non-stationary. In this setup, 200 random permutations are applied to the pixels of MNIST characters sequentially throughout the training process.

As shown in Figure 7, CPR demonstrates the ability to maintain plasticity in the supervised learning regime as well. This experiment tests over 200 tasks, more than any other experiment, and shows CPR matching the stability of reset-based baselines while significantly outperforming standard optimizers like Adam, which exhibit characteristic plasticity loss as the task sequence progresses. The similarity in performance in this experiment demonstrates the interplay between the number of tasks and their complexity when investigating plasticity loss.

E Rollout Size

Increasing rollout size in on-policy RL is known to stabilize training by reducing gradient approximation noise. While we confirm that sufficiently large rollouts can delay policy collapse in baselines like Adam, relying on massive trajectory buffers masks optimization pathologies rather than solving them. Specifically, we find that large rollouts merely dampen the rate of dormant neuron accumulation without preventing the underlying plasticity loss.

We intentionally adopt a smaller on-policy rollout size to rigorously model the memory and latency constraints of large-scale robotics and high-throughput RL. In these regimes, allocating VRAM to massive buffers is often impractical and competes with the need for high-frequency updates and sample efficiency. Consequently, algorithms that rely on large batches to maintain stability are brittle in resource-constrained environments. Our setup serves as a critical stress test: a robust continual learning algorithm must maintain plasticity even when high-variance gradient estimates are unavoidable.

Large rollout sizes reduce the variance of gradient estimation. We demonstrate this empirically in Figure 8 where we show how increasing the rollout size can mitigate the policy collapse problem in the SlipperyAnt environment. This motivates the use of dormant reset methods to directly address neuron dormancy under data non-stationarity.

F Hyperparameter Configuration

F.1 Continual Ant Experiment Hyperparameter Settings

The following table summarizes the hyperparameter search space and the optimal values found for each algorithm used in this project. Max reset fraction indicates a maximum proportion of units

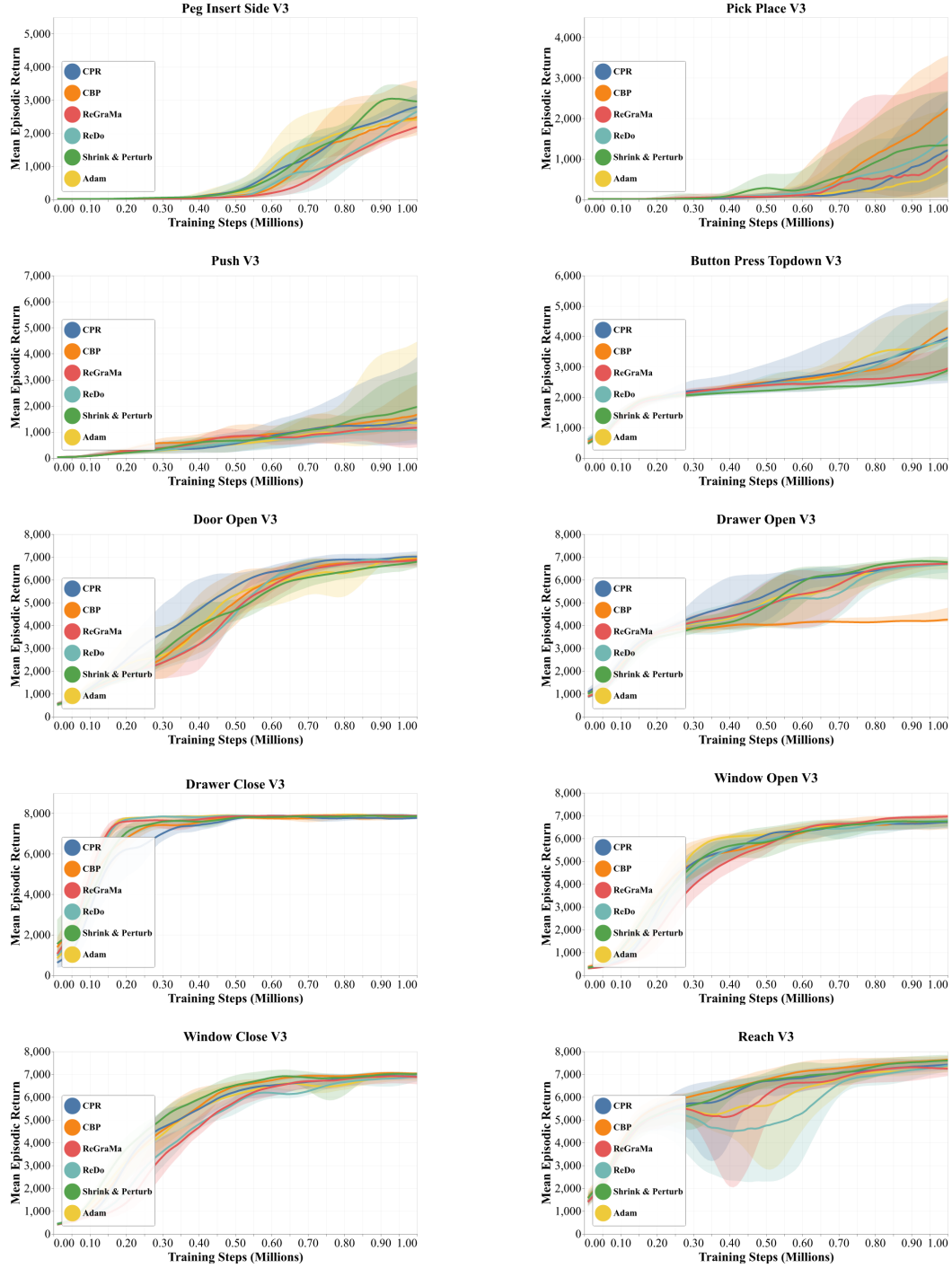


Figure 6: Mean episodic return over 2M steps for 10 MetaWorld continuous-control tasks (IQM over 10 seeds, shaded area indicates IQR)

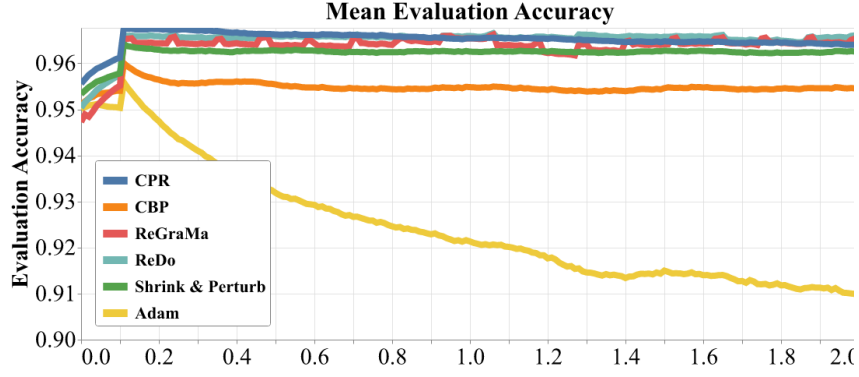


Figure 7: Permutated MNIST Continual Supervised Learning. The Interquartile Mean (IQM) of evaluation accuracy shows CPR maintaining high plasticity and adaptation efficiency throughout the 2 million step training horizon, preventing the performance degradation observed in standard Adam.

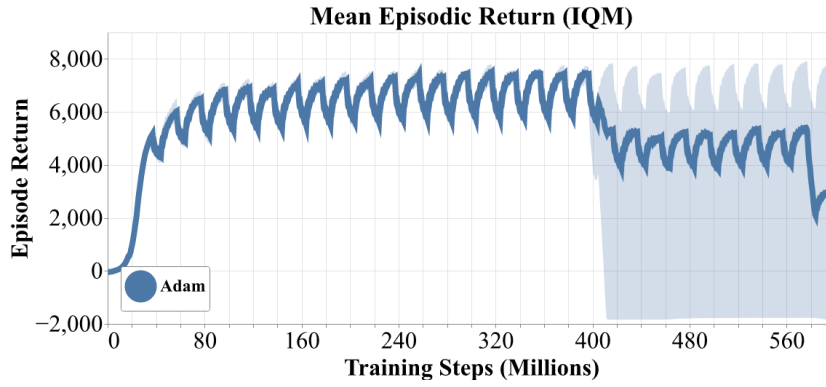
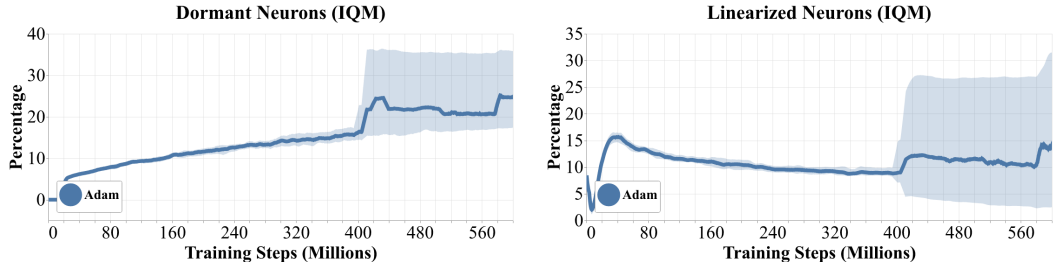


Figure 8: IQM of Adam using larger rollout, with IQR for shaded regions over 15 seeds. This demonstrates large rollouts can delay policy collapse in continual reinforcement learning training



(a) Dormant neurons for Adam with larger rollouts, with IQR for shaded regions over 15 seeds. Larger batch sizes delay dormant neuron accumulation. (b) IQM of linearised neurons using Adam and a larger rollout, with IQR for shaded regions over 15 seeds. Larger batch sizes delay linearised neuron accumulation.

Figure 9: Effect of larger rollouts on dormant and linearised neuron accumulation.

to be reset as is used by Sokar et al. (2023), where "None" indicates an uncapped reset proportion. Note, different seeds were used for sweeping hyperparameters than generating main results.

Table 3: PPO Hyperparameters

Parameter	Value
Learning Rate (SlipperyAnt)	10^{-3}
Learning Rate (SlipperyHumanoid)	3×10^{-4}
Rollout Steps (SlipperyAnt)	196,608
Rollout Steps (SlipperyHumanoid)	327,680
Number of Epochs	4
Gradient Steps	32
Discount Factor (γ)	0.97
GAE Lambda (λ)	0.95
Entropy Coefficient	10^{-3}
Clip Epsilon	0.2
Value Function Coefficient	0.5
Normalize Advantages	True
Number of Environments	2048
Number of Tasks	20
Episode Length	1000
Steps per Task	20,000,000

Table 4: PPO Network Architecture

Parameter	Policy Network	Value Function
Number of Layers	4	5
Hidden Size (SlipperyAnt)	32	256
Hidden Size (SlipperyHumanoid)	128	256
Output Size	8	1
Activation Function	Swish	Swish
Kernel Initialization	LeCun Normal	LeCun Normal
Data Type	float32	float32

F.2 Continual MinAtar Experiment Hyperparameter Settings

The Continual MinAtar experiments use discrete SAC with a CNN actor/critic and Adam as the base optimizer. The base Adam optimizer used a learning rate of 3×10^{-4} for each baseline. Table 8 reports the selected hyperparameters for each reset method, which were attached on top of Adam via the Optax chain described in Appendix H. The SAC optimization settings are the same as Table 6 except with a buffer size of 1M, replay ratio of 4, and 12 parallel envs. Additionally, Continual MinAtar uses a convolutional input layer with 3x3 kernels and stride 1, followed by a single 128-unit hidden layer before the output head.

F.3 Continual MetaWorld Experiment Hyperparameter Settings

The Continual MetaWorld experiments use SAC with Muon as the base optimizer. Muon used a learning rate of 10^{-4} for each baseline. Table 9 reports the selected hyperparameters for each reset method. The SAC hyperparameters are provided in Table 6.

Table 5: Hyperparameters swept for 5 seeds per configuration. Rankings are based on average final performance. Optimal values are reported as SlipperyAnt / SlipperyHumanoid and correspond to the values used for the main-paper training results.

Algorithm	Hyperparameter	Search Space	Selected Values
CPR	Decay Rate	[0.9, 0.99]	0.99 / 0.99
	Max Per-unit Reset	[0.01 to 0.05 (step 0.005; $N = 9$)]	0.015 / 0.05
	Update Frequency	[100, 1000, 10000]	1000 / 1000
CBP	Decay Rate	[0.9, 0.99]	0.99 / 0.99
	Replacement Rate	$[10^{-4}, 5 \times 10^{-4}, 10^{-3}, 2.5 \times 10^{-3}, 3 \times 10^{-3}, 4 \times 10^{-3}]$	$3 \times 10^{-3} / 2.5 \times 10^{-3}$
	Maturity Threshold	[100, 1000, 10000]	100 / 100
ReDo	Score Threshold	[0.05 to 0.75 (step 0.1; $N=7$)]	0.65 / 0.5
	Max Reset Fraction	[None, 0.02, 0.05]	None / None
	Update Frequency	[100, 1000, 10000]	100 / 100
ReGraMa	Score Threshold	[0.05 to 0.75 (step 0.1; $N=7$)]	0.25 / 0.15
	Max Reset Fraction	[None, 0.02, 0.05]	None / None
	Update Frequency	[100, 1000, 10000]	100 / 100
Shrink & Perturb	Shrink	$[10^{-5}, 10^{-4}, 10^{-3}, 5 \times 10^{-3}]$	$10^{-3} / 10^{-3}$
	Perturb	$[10^{-5}, 10^{-4}, 10^{-3}, 5 \times 10^{-3}, 10^{-2}]$	$5 \times 10^{-3} / 5 \times 10^{-3}$
	Interval	[100, 1000, 10000]	1000 / 1000

Table 6: Default SAC hyperparameters for Continual MetaWorld experiments

Parameter	Value
Learning starts	5,000
Replay ratio	8
Batch size	256
Buffer size	500,000
Hidden layers	3
Hidden size	256
Activation	ReLU
Dtype	float32
Discount (γ)	0.99
Soft update (τ)	0.005
Initial α	0.2
α learning rate	3×10^{-4}
Target entropy	$-d_{\text{action}}$
$\log \sigma$ bounds	$[-20, 2]$
Num envs	10

G Continuous Transformation choice

In the CPR algorithm, we map the utility score u_i^ℓ of a neuron to a partial reset fraction r_i^ℓ . This mapping is controlled by a transformation function $\phi : \mathbb{R} \rightarrow [0, 1]$, a sharpness parameter κ , a threshold (typically centered at the layer mean, $u = 1$), and a maximum per-unit reset fraction ρ .

Table 7: Hyperparameter sweep ranges and selected values for MetaWorld MT1 experiments, tuned over 5 seeds on Peg Insert Side

Method	Parameter	Search Space	Selected Value
CPR	Decay Rate	[0.9, 0.99]	0.99
	Max Per-unit Reset	[0.01, 0.015, 0.02, 0.025]	0.01
	Update Frequency	[1000, 5000, 10,000]	1,000
CBP	Replacement Rate	[10^{-5} , 10^{-4} , 10^{-3}]	10^{-5}
	Decay Rate	[0.9, 0.99, 0.999]	0.999
	Maturity Threshold	[1000, 5000, 10,000]	1,000
ReGraMa / ReDo	Update Frequency	[10^4 , 10^5]	10^5
	Score Threshold	[10^{-4} , 10^{-3} , 10^{-2}]	10^{-4}
	Max Reset Fraction	[None, 0.02, 0.05]	0.02
Shrink & Perturb	Shrink	[10^{-2} , 10^{-3} , 10^{-4}]	10^{-4}
	Perturbation Std	[10^{-3} , 10^{-2} , 10^{-1}]	10^{-3}
	Interval	[1000, 5000, 10,000]	1,000

Table 8: Selected reset-method hyperparameters for Continual MinAtar experiments.

Method	Parameter	Search Space	Selected Value
CPR	Decay Rate	[0.9, 0.99]	0.99
	Max Per-unit Reset	[0.01, 0.015, 0.02, 0.025]	0.015
	Update Frequency	[10^3 , 10^4 , 10^5]	10^3
CBP	Replacement Rate	[10^{-6} , 10^{-5} , 10^{-4}]	10^{-6}
	Decay Rate	[0.9, 0.99, 0.999]	0.99
	Maturity Threshold	[10^3 , 10^4 , 10^5]	10^3
ReGraMa	Update Frequency	[10^3 , 10^4 , 10^5]	10^5
	Score Threshold	[5×10^{-5} , 10^{-4} , 5×10^{-4}]	10^{-4}
	Max Reset Fraction	[None, 0.02, 0.05]	0.02
ReDo	Update Frequency	[10^3 , 10^4 , 10^5]	10^5
	Score Threshold	[5×10^{-5} , 10^{-4} , 5×10^{-4}]	10^{-4}
	Max Reset Fraction	[None, 0.02, 0.05]	0.02
Shrink & Perturb	Shrink	[10^{-3} , 10^{-4} , 10^{-5}]	10^{-5}
	Perturbation Std	[10^{-3} , 10^{-4} , 10^{-5}]	10^{-4}
	Interval	[10^3 , 10^4 , 10^5]	10^3

The update rule is defined as:

$$r_i^\ell = \rho \cdot \phi(u_i^\ell) \quad (8)$$

We evaluate four candidate shapes for $\phi(u)$. To ensure fair comparison, all functions are normalized such that $\phi(1) = 1$. That is, a neuron with utility exactly at the threshold receives the maximum reset penalty ρ . As utility increases, $\phi(u)$ decays toward 0.

The functions are defined as follows:

Table 9: Hyperparameter sweep ranges and selected values for Continual MetaWorld experiments, tuned over 5 seeds

Method	Parameter	Search Space	Selected Value
CPR	Decay Rate	[0.9, 0.99]	0.99
	Max Per-unit Reset	[0.01, 0.015, 0.02, 0.025]	0.015
	Update Frequency	[10^3 , 10^4 , 10^5]	10^3
CBP	Replacement Rate	[10^{-6} , 10^{-5} , 10^{-4}]	10^{-6}
	Decay Rate	[0.9, 0.99, 0.999]	0.99
	Maturity Threshold	[10^3 , 10^4 , 10^5]	10^4
ReGraMa	Update Frequency	[10^3 , 10^4 , 10^5]	10^5
	Score Threshold	[10^{-4} , 10^{-3} , 10^{-2}]	10^{-4}
	Max Reset Fraction	[None, 0.02, 0.05]	0.02
ReDo	Update Frequency	[10^3 , 10^4 , 10^5]	10^5
	Score Threshold	[10^{-4} , 10^{-3} , 10^{-2}]	10^{-4}
	Max Reset Fraction	[None, 0.02, 0.05]	0.02
Shrink & Perturb	Shrink	[10^{-3} , 10^{-4} , 10^{-5}]	10^{-5}
	Perturbation Std	[10^{-3} , 10^{-4} , 10^{-5}]	10^{-4}
	Interval	[10^3 , 10^4 , 10^5]	10^3

$$\phi_{\text{Exp}}(u) = \min(\exp[-\kappa(u-1)], 1) \quad (9)$$

$$\phi_{\text{Sigmoid}}(u) = \min(2 \cdot \sigma[-\kappa(u-1)], 1) \quad (10)$$

$$\phi_{\text{Softplus}}(u) = \min\left(\frac{\text{softplus}[\kappa(1-u)]}{\ln 2}, 1\right) \quad (11)$$

$$\phi_{\text{Linear}}(u) = \text{clip}(1 - \kappa(u-1), 0, 1) \quad (12)$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ is the logistic sigmoid, $\text{softplus}(z) = \ln(1 + e^z)$, and $\text{clip}(x, a, b) = \max(a, \min(x, b))$.

Normalization details:

- **Sigmoid:** We multiply by 2 because $\sigma(0) = 0.5$. At the mean utility $u = 1$, the argument is 0, resulting in $2 \cdot 0.5 = 1$.
- **Softplus:** We divide by $\ln 2$ because $\text{softplus}(0) = \ln(1 + e^0) = \ln 2$. This ensures that at $u = 1$, the fraction simplifies to 1.

Transformation	Peak	Average	Final
Linear	8223.515	5778.549	5873.256
Exponential	8326.043	5999.879	7598.191
Softplus	8145.749	5917.624	6896.087
Sigmoid	8360.509	6066.286	7498.798

Table 10: Training performance on SlipperyAnt (IQM across 15 seeds) with $\rho = 0.015$ and $\kappa = 16$.

As shown in Table 10, the Sigmoid and Exponential transformations yield the highest returns. To ensure a fair comparison, we calibrated each function to share the same threshold value and fixed $\rho = 0.015$.

We observe that the choice of transformation shape has a measurable impact on performance. The Exponential, Sigmoid, and Softplus functions all amplify resets for neurons with *moderately* low

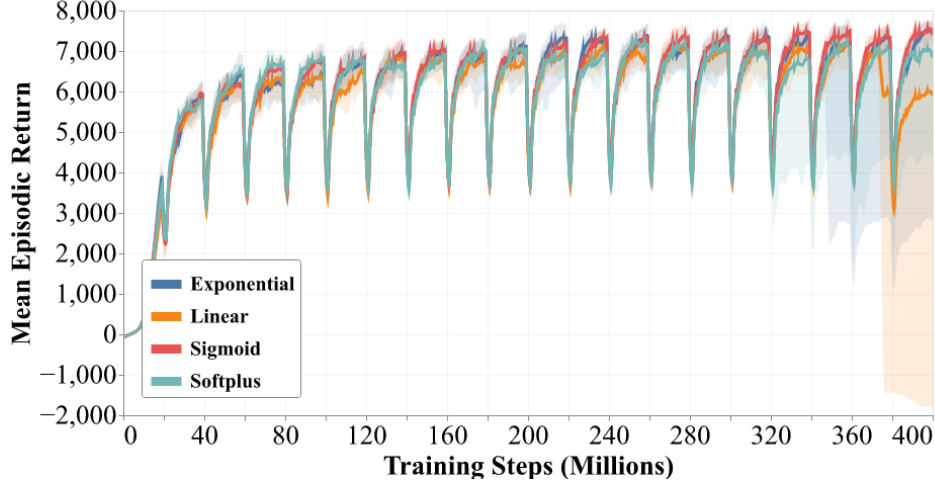


Figure 10: Utility transformation function comparison

gradients, whereas the Linear function is too conservative. The Sigmoid and Exponential functions perform best because their non-linear decay aligns well with the distribution of neuron gradients near the mean; this strengthens the claim for a non-linear transformation.

CPR Utility Distribution Throughout Training

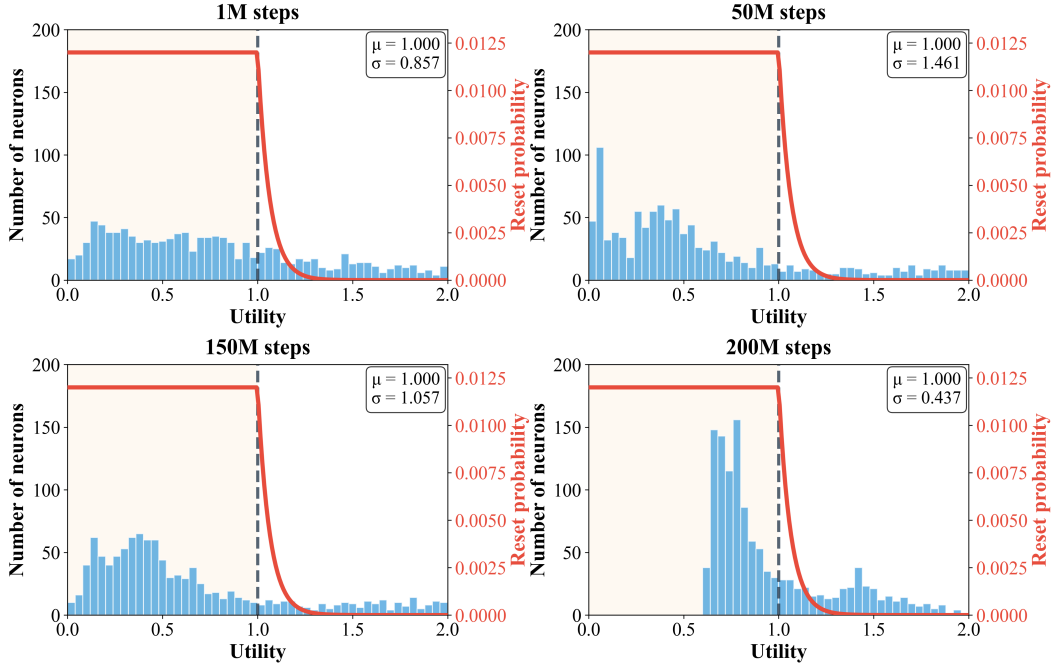


Figure 11: Results from a SlippyAnt run showing that CPR pushes neurons away from dormant regions

H Implementation Details

Compatibility with optimizers. Other neuron-reset methods additionally reset underlying optimizer state, such as μ and ν parameters in Adam (Kingma & Ba, 2014), for the affected low-utility

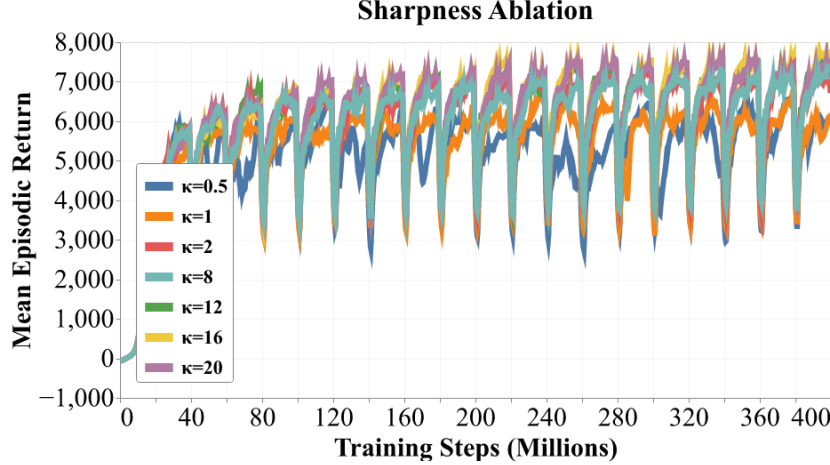


Figure 12: Ablation of the sharpness parameter (κ) on the SlippyAnt environment. Performance is highly consistent across a wide range of values (κ), indicating that CPR is robust to hyperparameter tuning. Notably, low sharpness values ($\kappa \leq 1$) lead to significant performance degradation, confirming that resets must be targeted at low-utility neurons rather than applied indiscriminately.

units. We did not find base optimizer parameter resets necessary for our method, which reduces overall implementation complexity.

SlippyAnt and SlippyHumanoid. For scalable environment parallelism, we build these continual RL environments in Brax (Freeman et al., 2021), a JAX (Bradbury et al., 2018) based physics engine. We implement all baselines and CPR in JAX, which allows compilation of the Continual RL loop end-to-end for faster experimentation, with individual seeds taking approximately an hour on a single Nvidia RTX3090 GPU.

Continual MetaWorld. Inspired by ContinualWorld (Wolczyk et al., 2021) we iterate through MetaWorld (McLean et al., 2025) tasks sequentially. While ContinualWorld has a focus on catastrophic forgetting we instead measure the adaptation performance using the latest versions of MetaWorld environments. We use the average and final performance of plasticity preservation baselines on this benchmark. This benchmark would take 3 days to run using an RTX3090.

Continual MinAtar. The Continual MinAtar benchmark was proposed by Tang et al. (2025) and iterates through three discrete, vision-based Atari tasks (Space Invaders, Asterix and Seaquest). For this benchmark we use discrete SAC with a CNN head for actor and value networks. This benchmark would take 4 hours to run using an RTX3090.

Normalization and gating. We form a normalized score $\tilde{u}_i^\ell(t)$ within layer ℓ to make selection thresholds comparable across layers and time.

Reset operator. When resetting a unit i in layer ℓ :

1. **Incoming weights:** resample the incoming row $W_{i,:}^\ell$ (and bias b_i^ℓ if present) from the layer’s initial distribution (e.g., Kaiming/He).
2. **Outgoing weights:** scale the outgoing column $W_{:,i}^{\ell+1}$ towards *zero*. This prevents immediate disruption to downstream computations and is the standard reset-method design choice, validated in prior work (Sokar et al., 2023; Dohare et al., 2024).

Reproducibility Statement. All of our code, including baselines, environments, experiments, and plotting scripts, is available at: github.com/LucMc/continual-learning/ and open-source under an

MIT license. To facilitate reproducibility, the code provides declarative experiment files for generating figures and all project dependencies are specified in the `pyproject.toml` file at the root of the project, with a `uv.lock` file pinning them to specific versions used. The code also provides a comprehensive `README.md` which offers step-by-step instructions for reproducing our results and figures. Finally, we provide full details of hyperparameters used in [Appendix F](#).

Optimizer and Optax chain. Our implementation treats reset-methods as optimizer wrappers. We provide an easy-to-use interface for attaching reset methods to optimizer pipelines. Our code is implemented in JAX ([Bradbury et al., 2018](#)) with Optax ([DeepMind et al., 2020](#)).

Once attached, the Optax optimizer can be used in the same way as a regular Optax optimizer, only if the reset method requires features (i.e. ReDo or CBP) then these are taken as an input to the `optimizer.update` function.

A simple optimizer setup is shown in [Listing 1](#)

Listing 1: Demonstration of simple optimizer using a reset method such as CPR

```
from continual_learning.optim import CPR
from continual_learning.utils import attach_reset_method
import optax

# tx can be simple
tx = optax.adam()

# or composite
tx = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.adam()
)

# ready to be used like any other optax optimizer
tx_w_reset = attach_reset_method(
    ("tx", tx),
    ("reset_method", CPR())
)
```

I Additional Experiments

Table 11: SlipperyAnt average over training, final return (measured at 400M steps for all methods), and peak return. Entries show point estimate with (+ / -) IQR. Best final and best peak are in **bold**.

Method	IQM Return	Final Return (at 400M)	Peak Return
CPR	6051 (+290/ - 320)	7494 (+230/ - 235)	8380 (+68/ - 131)
CBP	4932 (+576/ - 873)	5643 (+971/ - 2723)	7495 (+426/ - 787)
ReDo	3470 (+1038/ - 2434)	3883 (+1621/ - 5731)	6459 (+492/ - 4437)
ReGraMa	3741 (+1138/ - 2717)	4635 (+1387/ - 6458)	6804 (+316/ - 558)
Shrink			
& Perturb	3469 (+2733/ - 2902)	-341 (+7820/ - 1656)	7812 (+297/ - 248)
Adam	1958 (+1465/ - 923)	-1921 (+92/ - 77)	8034 (+177/ - 268)

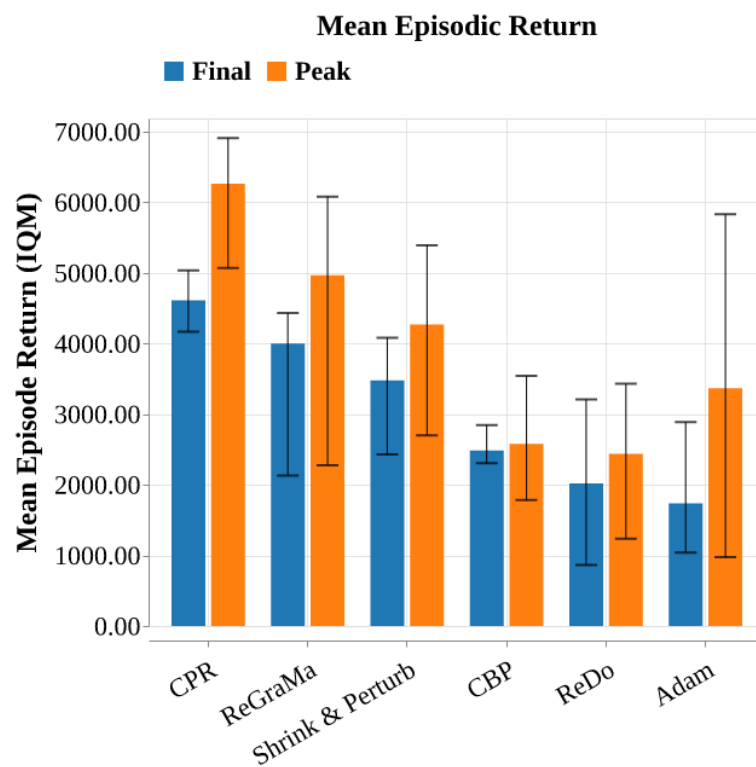


Figure 13: Peak vs. final IQM episodic return in SlipperyHumanoid (higher is better).

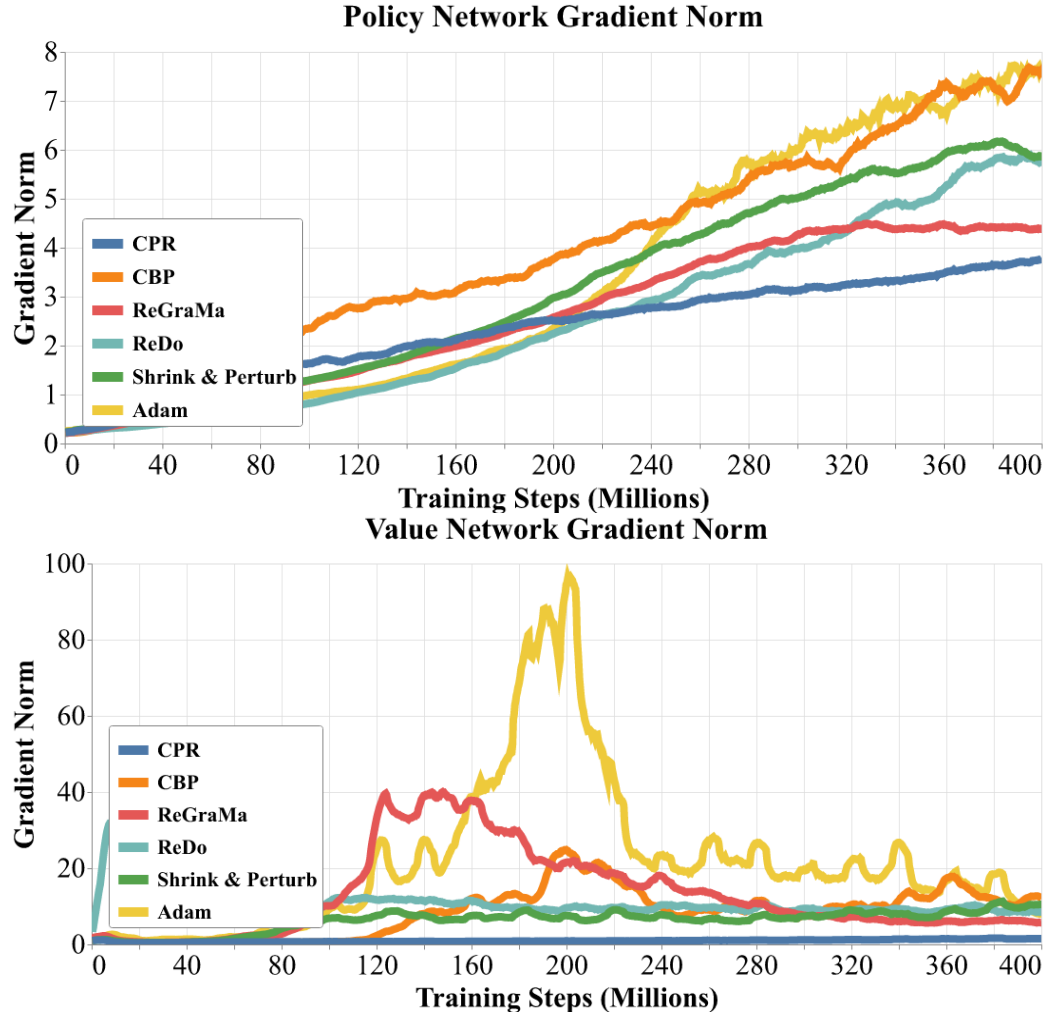


Figure 14: The gradient norm for SlipperyHumanoid, averaged over both actor and critic with IQM across 15 seeds. **Top:** Policy Network **Bottom:** Value Network. This demonstrates CPR maintains stable gradient norm throughout training

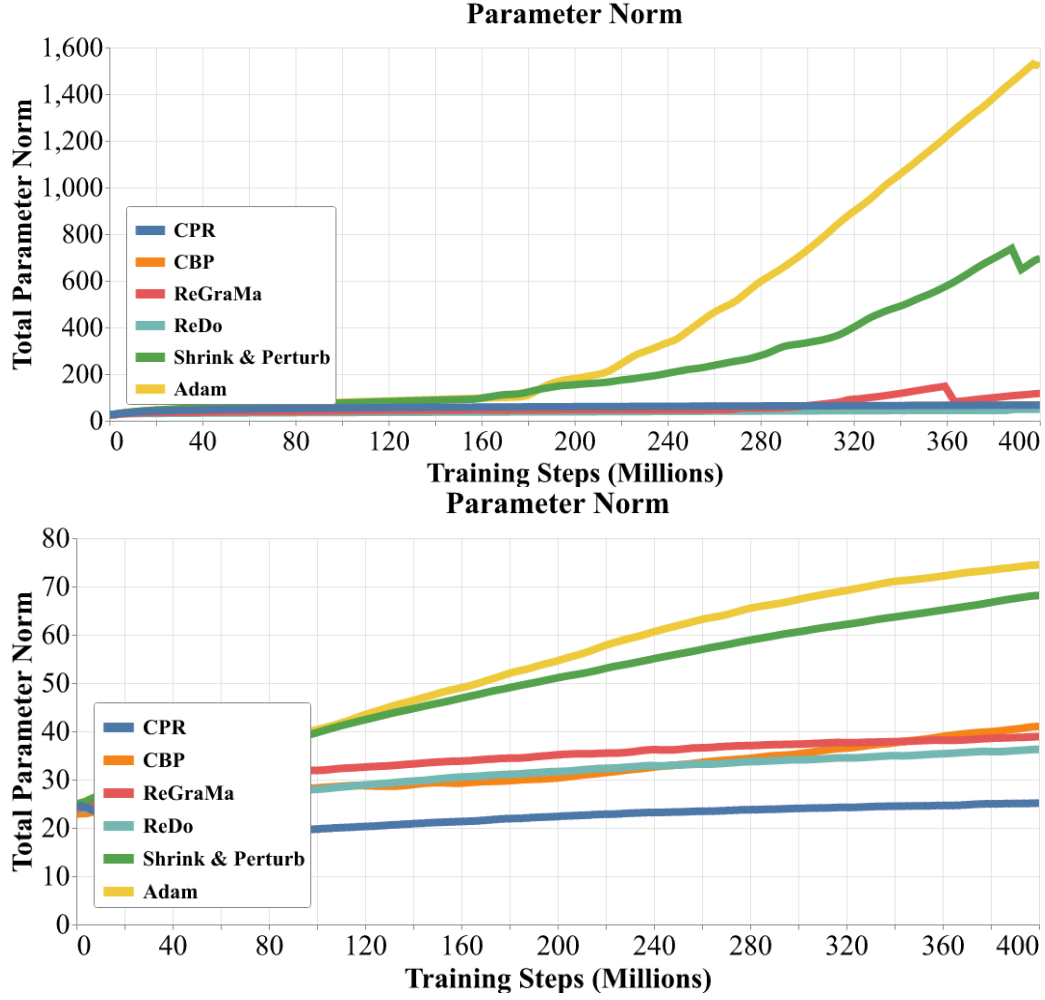


Figure 15: The parameter norm for **Top: SlippyAnt** **Bottom: SlippyHumanoid**, averaged over both actor and critic with IQM across 15 seeds. This demonstrates that reset methods can prevent parameter-norm growth by periodically refreshing low-utility units.

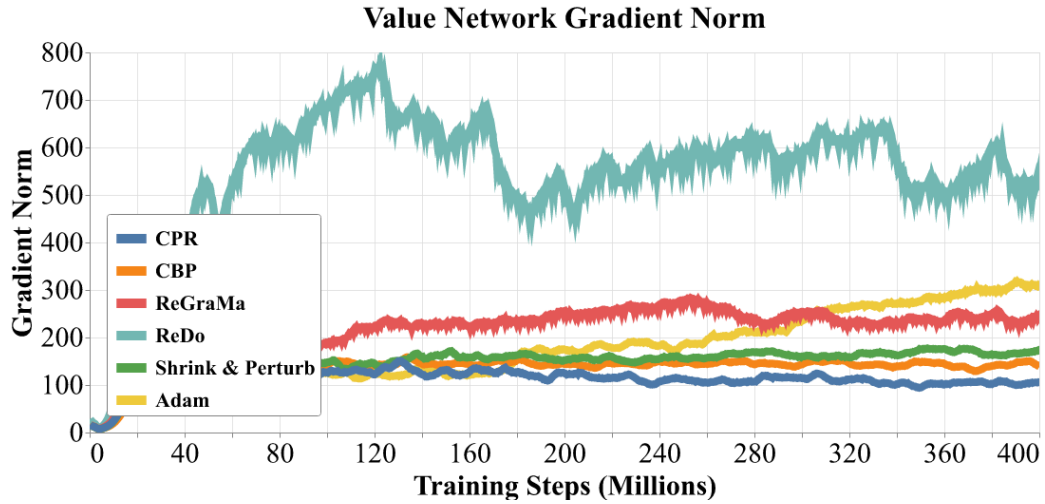


Figure 16: The value network gradient norm for SlippyAnt, averaged with IQM across 15 seeds.

Table 12: SlipperyHumanoid average over training, final return (measured at 400M steps for all methods), and peak return with the step at which it occurs. Entries show point estimate with (+ / -) IQR. Best final and best peak are in **bold**.

Method	Average Return	Final Return (at 400M)	Peak Return
CPR	3270 (+488/ - 465)	4611 (+425/ - 443)	6264 (+647/ - 1195) at 382M steps
CBP	1890 (+365/ - 309)	2487 (+359/ - 181)	2581 (+962/ - 797) at 222M steps
ReDo	1340 (+646/ - 554)	2021 (+1189/ - 1158)	2439 (+992/ - 1203) at 322M steps
ReGraMa	2393 (+779/ - 1180)	4001 (+431/ - 1870)	4967 (+1114/ - 2690) at 382M steps
Shrink			
& Perturb	2440 (+652/ - 671)	3477 (+603/ - 1049)	4268 (+1123/ - 1567) at 282M steps
Adam	1666 (+994/ - 838)	1737 (+1149/ - 697)	3366 (+2465/ - 2390) at 262M steps

Table 13: Runtime comparison on SlipperyHumanoid. All methods used an identical training setup over 15 seeds on a Nvidia RTX3090. As our implementation captures activations regardless of the reset method, most methods operate at similar computational complexity. The exception being CBP which performs age tracking and reset logic each step.

Method	Median (h)	Overhead
Adam	1.17	—
ReGraMa	1.22	+4%
Shrink & Perturb	1.23	+5%
CPR	1.24	+6%
ReDo	1.25	+7%
CBP	1.34	+15%